

**Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação
Disciplina de Organização de Arquivos (SCC0215)**

Docente

Profa. Dra. Cristina Dutra de Aguiar
cdac@icmc.usp.br

Monitores

Gustavo Ramos Santos Pires

gustavo.rspires@usp.br ou telegram: @darrocaxd

João Gabriel Pieroli da Silva

joaogabrielpieroli@usp.br ou telegram: @bielcomp24

Pedro Lunkes Villela

pedrolunkesvillela@usp.br ou telegram: @pedrolunkes

Renan Banci Catarin

renanbcatarin@usp.br ou telegram: @Reckat

Renan Trofino Silva

renan.trofino@usp.br ou telegram: @renan823

Vinícius Souza Freitas

vininicius@usp.br ou telegram: @Vininicius

Trabalho Prático 2

Este trabalho tem como objetivo realizar a junção entre dois arquivos de dados.

*O trabalho deve ser feito por, no máximo, **2 alunos da mesma turma**. Os alunos devem ser os mesmos do trabalho introdutório. Quaisquer mudanças devem ser devidamente informadas. A solução deve ser proposta exclusivamente pelo(s) aluno(s) com base nos conhecimentos adquiridos nas aulas. Consulte as notas de aula e o livro texto quando necessário.*

Fundamentos da disciplina de Bases de Dados

A disciplina de Organização de Arquivos é uma disciplina fundamental para a disciplina de Bases de Dados. A definição dos trabalhos práticos é feita considerando esse aspecto, ou seja, os trabalhos são especificados em termos de várias funcionalidades, e essas funcionalidades são relacionadas tanto com desafios enfrentados no mercado de trabalho quanto com as funcionalidades da linguagem SQL (*Structured Query Language*), que é a linguagem utilizada por sistemas gerenciadores de banco de dados (SGBDs) relacionais. As características e o detalhamento de SQL

serão aprendidos na disciplina de Bases de Dados. Porém, por meio do desenvolvimento deste trabalho prático, os alunos podem entrar em contato com alguns comandos da linguagem SQL e verificar como eles são implementados.

Este trabalho prático tem como objetivo implementar a operação de junção. Ela é amplamente utilizada em aplicações de banco de dados desde que ela serve para combinar registros de arquivos diferentes usando como base um campo de igualdade (condição de junção), produzindo um único arquivo integrado.

Considere um primeiro arquivo de dados `arquivoA` com os campos `Acampo1`, `Acampo2`, ..., `AcampoN`. Considere um segundo arquivo de dados `arquivoB` com os campos `Bcampo1`, `Bcampo2`, ..., `BcampoM`. As seguintes situações podem ocorrer na junção de `arquivoA` e `arquivoB` considerando a condição de junção `Acampo1 = Bcampo1`:

- Não existe igualdade entre `Acampo1` e `Bcampo1`. Nesse caso, nenhum registro é gerado como resultado da junção.
- Existe apenas uma igualdade entre `Acampo1` e `Bcampo1`. Nesse caso, somente um registro é gerado como resultado da junção.
- Existem k igualdades entre `Acampo1` e `Bcampo1`. Nesse caso, são gerados k registros como resultado da junção.

Programa

Descrição Geral. Implemente um programa em C por meio do qual seja possível realizar a junção de dois arquivos de dados, considerando diferentes formas de implementação dessa operação.

Importante. A definição da sintaxe de cada comando bem como sua saída devem seguir estritamente as especificações definidas em cada funcionalidade. Para especificar a sintaxe de execução, considere que o programa seja chamado de “programaTrab”. Essas orientações devem ser seguidas uma vez que a correção do funcionamento do programa

se dará de forma automática. De forma geral, a primeira entrada da entrada padrão é sempre o identificador de suas funcionalidades, conforme especificado a seguir.

Modularização. É importante modularizar o código. Trechos de programa que aparecerem várias vezes devem ser modularizados em funções e procedimentos.

Descrição Específica. O programa deve oferecer as seguintes funcionalidades:

Na linguagem SQL, o comando SELECT é usado para listar os dados de uma ou mais tabelas (arquivos de dados). Quando mais do que uma tabela é usada, essas tabelas são "juntadas" por meio de uma (ou mais) coluna(s) de junção. Existem duas diferentes formas de se especificar a junção no comando SELECT:

SELECT lista de colunas (ou seja, campos a serem exibidos na resposta)

FROM tabela1, tabela2 (ou seja, arquivos que contêm os campos)

(podem existir mais tabelas)

WHERE tabela1.nomeCampo = tabela2.nomeCampo (condição de junção)

AND critério de seleção (ou seja, critério de busca)

ou

SELECT lista de colunas (ou seja, campos a serem exibidos na resposta)

FROM tabela1 JOIN tabela2 ON tabela1.nomeCampo = tabela2.nomeCampo

(ou seja, arquivos que contêm os campos + condição de junção)

(podem existir mais tabelas)

WHERE critério de seleção (ou seja, critério de busca)

[11] Permita a recuperação dos dados de todos os registros armazenados no arquivo de dados **estacao.bin** considerando o campo **codProxEstacao**, juntando-os de forma apropriada com os dados de **estacao.bin** considerando o campo **codEstacao**. Essa funcionalidade requer a realização de uma autojunção, ou seja, o mesmo arquivo é usado duas vezes na junção, considerando como condição de junção **estacao1.codProxEstacao = estacao2.codEstacao**.

Existem várias formas de se implementar a junção e, nesta funcionalidade, ela deve ser implementada por meio da **junção de loop aninhado**. A junção de loop aninhado é um algoritmo padrão (de força bruta), desde que não exige o uso de índices ou outras melhorias em qualquer arquivo para realizar a junção.

O algoritmo da **junção de loop aninhado** é definido como segue. Para cada registro presente no arquivo **arquivoA** (loop externo), recupere cada registro do arquivo **arquivoB** (loop interno) e teste se os dois registros satisfazem à condição de junção **Acampo1 = Bcampo1**. Ou seja:

```
para cada registro em estacao1 faça      // loop externo
    para cada registro em estacao2 faça  // loop interno
        se estacao1.codProxEstacao = estacao2.codEstacao
            então mostre os campos desejados de estacao1 e estacao2
        fim-se
    fim-para
fim-para
```

Entrada do programa para a funcionalidade [11]:

```
11 arquivoEntrada1.bin nomeCampo1 arquivoEntrada2.bin nomeCampo2
```

onde:

- arquivoEntrada1.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos.
- nomeCampo1 é o nome do campo do arquivo de dados arquivoEntrada1.bin que é usado como condição de junção. Neste trabalho prático, apenas codProxEstacao pode ser utilizado.
- arquivoEntrada2.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos.
- nomeCampo2 é o nome do campo do arquivo de dados arquivoEntrada2.bin que é usado como condição de junção. Neste trabalho prático, apenas codEstacao pode ser utilizado.

Saída caso o programa seja executado com sucesso:

Para cada registro do arquivo estacao.bin, mostre os valores dos campos codEstacao, nomeEstacao, nomeLinha, codProxEstacao, nomeProxEstacao, sendo o valor de nomeProxEstacao obtido por meio da junção. Os valores desses campos devem ser mostrados de forma sequencial em uma única linha, separados por espaço em branco. Ver exemplo ilustrado no **exemplo de execução**.

Mensagem de saída caso não seja gerado nenhum registro na junção dos dois arquivos:

Registro inexistente.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
11 estacao1.bin codProxEst estacao2.bin codEstacao
1 Tucuruvi Azul 2 Parada Inglesa
2 Parada Inglesa Azul 3 Jardim Sao Paulo
3 Jardim Sao Paulo Azul 4 Santana
...
```

[12] Permita a recuperação dos dados de todos os registros armazenados no arquivo de dados **estacao.bin** considerando o campo `codProxEstacao`, juntando-os de forma apropriada com os dados de **estacao.bin** considerando o campo `codEstacao`. Essa funcionalidade requer a realização de uma autojunção, ou seja, o mesmo arquivo é usado duas vezes na junção, considerando como condição de junção **`estacao1.codProxEstacao = estacao2.codEstacao`**.

Existem várias formas de se implementar a junção e, nesta funcionalidade, ela deve ser implementada por meio da **junção de loop único**. A junção de loop único depende da existência de índices. Portanto, o índice criado no trabalho prático 1 deve ser usado neste trabalho.

O algoritmo da **junção de loop único** é definido como segue. Para cada registro presente no arquivo `arquivoA` (loop externo), recupere cada registro do arquivo `arquivoB` (loop interno) usando o índice e teste se os dois registros satisfazem à condição de junção `Acampo1 = Bcampo1`. Ou seja:

para cada registro em `estacao1` faça

selecione os registros de `estacao2` que satisfaçam à condição

`estacao1.codProxEstacao = estacao2.codEstacao`

usando o índice árvore-B definido sobre o arquivo `estacao2`

mostre os campos desejados de `estacao1` e `estacao2`

fim-para

Entrada do programa para a funcionalidade [12]:

```
12 arquivoEntrada1.bin nomeCampo1 arquivoEntrada2.bin nomeCampo2
indiceArquivoEntrada2.bin
```

onde:

- arquivoEntrada1.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos.
- nomeCampo1 é o nome do campo do arquivo de dados arquivoEntrada1.bin que é usado como condição de junção. Neste trabalho prático, apenas codProxEstacao pode ser utilizado.
- arquivoEntrada2.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos.
- nomeCampo2 é o nome do campo do arquivo de dados arquivoEntrada2.bin que é usado como condição de junção. Neste trabalho prático, apenas codEstacao pode ser utilizado.
- indiceArquivoEntrada2.bin é o arquivo binário de índice árvore-B que foi gerado conforme as especificações descritas no trabalho prático 1 e que indexa o arquivo de dados arquivoEntrada2.bin.

Saída caso o programa seja executado com sucesso:

Para cada registro do arquivo estacao.bin, mostre os valores dos campos codEstacao, nomeEstacao, nomeLinha, codProxEstacao, nomeProxEstacao, sendo o valor de nomeProxEstacao obtido por meio da junção. Os valores desses campos devem ser mostrados de forma sequencial em uma única linha, separados por espaço em branco. Ver exemplo ilustrado no **exemplo de execução**.

Mensagem de saída caso não seja gerado nenhum registro na junção dos dois arquivos:

Registro inexistente.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
12 estacao1.bin codProxEst estacao2.bin codEstacao indiceEstacao2.bin
1 Tucuruvi Azul 2 Parada Inglesa
2 Parada Ingesa Azul 3 Jardim Sao Paulo
3 Jardim Sao Paulo Azul 4 Santana
...
```

Na linguagem SQL, a cláusula ORDER BY é usada para ordenar os resultados de uma consulta em ordem crescente (padrão) ou decrescente, com base em uma ou mais colunas. Ela é especificada juntamente com o comando SELECT.

SELECT lista de colunas (ou seja, campos a serem exibidos na resposta)

FROM tabela (ou seja, arquivo)

WHERE ... // opcional

ORDER BY lista de colunas (ou seja, campos a serem ordenados na resposta)

A funcionalidade [13] representa um exemplo de implementação da cláusula ORDER BY.

[13] Ordene um arquivo de dados no formato binário de acordo com um campo de ordenação que não contém valores repetidos. Considere que a ordenação deve ser feita de forma crescente. Considere também que, para os campos que podem assumir valores nulos, o valor não nulo deve aparecer antes do que o valor nulo. A ordenação deve ser implementada considerando que o arquivo de dados cabe totalmente em memória primária (RAM). Portanto, o arquivo deve ser: (i) lido inteiramente do disco para a RAM; (ii) ordenado de forma crescente de acordo com a chave de ordenação usando-se qualquer algoritmo de ordenação disponível na biblioteca da linguagem C; e (iii) escrito inteiramente para disco novamente, gerando um novo arquivo de dados no formato binário, o qual encontra-se ordenado com base no campo de ordenação. O arquivo de dados original não deve ser removido.

Entrada do programa para a funcionalidade [13]:

13 arquivoEntrada.bin campoOrdenacao arquivoOrdenado.bin

onde:

- arquivoEntrada.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos.
- campoOrdenacao é um campo do arquivoEntrada.bin que é usado para ordenar o arquivo de dados. Neste trabalho, apenas os campos codProxEstacao e codEstacao podem ser utilizados como campos de ordenação.
- arquivoOrdenado.bin é um arquivo binário que é gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados ordenados e não contém registros logicamente removidos.

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados ordenado no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

./programaTrab

13 estacao.bin codEstacao estacaoOrdenado.bin

usar a função binarioNaTela antes de terminar a execução da funcionalidade, para mostrar a saída do arquivo binário estacaoOrdenado.bin.

[14] Permita a recuperação dos dados de todos os registros armazenados no arquivo de dados **estacao.bin** considerando o campo `codProxEstacao`, juntando-os de forma apropriada com os dados de **estacao.bin** considerando o campo `codEstacao`. Essa funcionalidade requer a realização de uma autojunção, ou seja, o mesmo arquivo é usado duas vezes na junção, considerando como condição de junção **`estacao1.codProxEstacao = estacao2.codEstacao`**.

Existem várias formas de se implementar a junção e, nesta funcionalidade, ela deve ser implementada por meio da **junção ordenação-intercalação**. A junção ordenação-intercalação é possível de ser utilizada quando os registros presentes em ambos `arquivoA` e `arquivoB` estão ordenados pelo valor dos atributos da condição de junção, ou seja, `Acampo1` e `Bcampo1`, respectivamente.

O algoritmo da **junção ordenação-intercalação** é definido como segue. Ordene cada arquivo de dados com base no atributo da condição de junção. Percorra os dois arquivos simultaneamente na ordem dos atributos de junção, combinando os registros que têm os mesmos valores para a condição de junção. Quando os arquivos encontram-se ordenados, esse é o algoritmo mais eficiente. Ou seja:

ordene o arquivo `estacao1` considerando o campo `codEstacao`

ordene o arquivo `estacao2` considerando o campo `codProxEstacao`

realize o **merge** dos arquivos `estacao1` e `estacao2` ordenados, satisfazendo à

condição `estacao1.codProxEstacao = estacao2.codEstacao`

mostre os campos desejados de `estacao1` e `estacao2`

Entrada do programa para a funcionalidade [14]:

```
14 arquivoEntrada1.bin nomeCampo1 arquivoEntrada2.bin nomeCampo2
```

onde:

- arquivoEntrada1.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos. Antes de realizar o merge dos arquivos, arquivoEntrada1.bin deve ser ordenado usando a funcionalidade [13] com codProxEstacao como campo de ordenação.
- nomeCampo1 é o nome do campo do arquivo de dados arquivoEntrada1.bin que é usado como condição de junção. Neste trabalho prático, apenas codProxEstacao pode ser utilizado.
- arquivoEntrada2.bin é um arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório, e que contém dados desordenados e registros logicamente removidos. Antes de realizar o merge dos arquivos, arquivoEntrada2.bin deve ser ordenado usando a funcionalidade [13] com codEstacao como campo de ordenação.
- nomeCampo2 é o nome do campo do arquivo de dados arquivoEntrada2.bin que é usado como condição de junção. Neste trabalho prático, apenas codEstacao pode ser utilizado.

Saída caso o programa seja executado com sucesso:

Para cada registro do arquivo estacao.bin, mostre os valores dos campos codEstacao, nomeEstacao, nomeLinha, codProxEstacao, nomeProxEstacao, sendo o valor de nomeProxEstacao obtido por meio da junção. Os valores desses campos devem ser mostrados de forma sequencial em uma única linha, separados por espaço em branco. Ver exemplo ilustrado no **exemplo de execução**.

Mensagem de saída caso não seja gerado nenhum registro na junção dos dois arquivos:

Registro inexistente.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
14 estacao1.bin codProxEst estacao2.bin codEstacao indiceEstacao2.bin
1 Tucuruvi Azul 2 Parada Inglesa
2 Parada Inglesa Azul 3 Jardim Sao Paulo
3 Jardim Sao Paulo Azul 4 Santana
...
```

Restrições

As seguintes restrições têm que ser garantidas no desenvolvimento do trabalho.

[1] O arquivo de dados deve ser gravado em disco no **modo binário**. O modo texto não pode ser usado.

[2] Os dados do registro descrevem os nomes dos campos, os quais não podem ser alterados. Ademais, todos os campos devem estar presentes na implementação, e nenhum campo adicional pode ser incluído. O tamanho e a ordem de cada campo deve obrigatoriamente seguir a especificação.

[3] Deve haver a manipulação de valores nulos, conforme as instruções definidas.

[4] Não é necessário realizar o tratamento de truncamento de dados.

[5] Devem ser exibidos avisos ou mensagens de erro de acordo com a especificação de cada funcionalidade.

[6] Os dados devem ser obrigatoriamente escritos campo a campo. Ou seja, não é possível escrever os dados registro a registro. Essa restrição refere-se à entrada/saída, ou seja, à forma como os dados são escritos no arquivo.

[7] O(s) aluno(s) que desenvolveu(desenvolveram) o trabalho prático deve(m) constar como comentário no início do código (i.e. NUSP e nome do aluno). Para trabalhos desenvolvidos por mais do que um aluno, não será atribuída nota ao aluno cujos dados não constarem no código fonte.

[8] Todo código fonte deve ser documentado. A **documentação interna** inclui, dentre outros, a documentação de procedimentos, de funções, de variáveis, de partes do código

fonte que realizam tarefas específicas. Ou seja, o código fonte deve ser documentado tanto em nível de rotinas quanto em nível de variáveis e blocos funcionais.

[9] A implementação deve ser realizada usando a linguagem de programação C. As funções das bibliotecas `<stdio.h>` devem ser utilizadas para operações relacionadas à escrita e leitura dos arquivos. A implementação não pode ser feita em qualquer outra linguagem de programação. O programa executará no [run.codes].

Material para Entregar

Arquivo compactado (a ser entregue no *run.codes*)

Deve ser preparado um arquivo .zip contendo:

- Código fonte do programa devidamente documentado.
- Makefile para a compilação do programa.

Vídeo (a ser entregue no *e-disciplinas*)

- Um vídeo gravado pelos integrantes do grupo, o qual deve ter, no máximo, 7 minutos de gravação. O vídeo deve explicar o trabalho desenvolvido. Ou seja, o grupo deve apresentar: cada funcionalidade e uma breve descrição de como a funcionalidade foi implementada. Todos os integrantes do grupo devem participar do vídeo, sendo que o tempo de apresentação dos integrantes deve ser balanceado. Ou seja, o tempo de participação de cada integrante deve ser aproximadamente o mesmo. O uso da webcam é obrigatório.

Instruções para fazer o arquivo makefile. No [run.codes] tem uma orientação para que, no makefile, a diretiva “all” contenha apenas o comando para compilar seu programa e, na diretiva “run”, apenas o comando para executá-lo. Adicionalmente, para utilizar a função `binarioNaTela`, é necessário usar a flag `-lmd`. Assim, a forma mais simples de se fazer o arquivo makefile é:

```
all:
    gcc -o programaTrab *.c -lmd
```

run:

`./programaTrab`

Lembrando que *.c já engloba todos os arquivos .c presentes no seu zip. Adicionalmente, no arquivo Makefile é importante se ter um *tab* nos locais colocados acima, senão ele pode não funcionar.

Instruções de entrega.

O programa deve ser submetido via [run.codes]:

- página: <https://runcodes.icmc.usp.br/>
- **Segunda Feira:** código de matrícula: **TZQ3**
- **Terça Feira:** código de matrícula: **71W4**

O vídeo gravado deve ser submetido por meio da página da disciplina no e-disciplinas, no qual o grupo vai informar o nome de cada integrante, o número do grupo e um link que contém o vídeo gravado. Ao submeter o link, verifique se o mesmo pode ser acessado. Vídeos cujos links não puderem ser acessados receberão nota zero. Vídeos corrompidos ou que não puderem ser corretamente acessados receberão nota zero.

Critério de Correção

Critério de avaliação do trabalho. Na correção do trabalho, serão ponderados os seguintes aspectos.

- Corretude da execução do programa.
- Atendimento às especificações do registro de cabeçalho e dos registros de dados.
- Atendimento às especificações da sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade.
- Qualidade da documentação entregue. A documentação interna terá um peso considerável no trabalho.
- Estruturação da solução. O projeto deve apresentar uma organização adequada de arquivos e diretórios.

- Modularização. Trechos de código que aparecem repetidamente devem ser modularizados por meio de funções ou procedimentos, evitando duplicação de código.
- Uso de ferramentas de inteligência artificial. Não deve ser realizado para a implementação das funcionalidades especificadas neste trabalho e outras funções julgadas como necessárias para o embasamento do conteúdo ministrado. O uso dessas ferramentas acarretará desconto rigoroso na nota.
- Vídeo. Integrantes que não participarem da apresentação receberão nota 0 no trabalho correspondente.

Casos de teste no [run.codes]. Juntamente com a especificação do trabalho, serão disponibilizados 70% dos casos de teste no [run.codes], para que os alunos possam avaliar o programa sendo desenvolvido. Os 30% restantes dos casos de teste serão utilizados nas correções.

Restrições adicionais sobre o critério de correção.

- A não execução de um programa devido a erros de compilação implica que a nota final da parte do trabalho será igual a zero (0).
- O não atendimento às especificações do registro de cabeçalho e dos registros de dados implica que haverá uma diminuição expressiva na nota do trabalho.
- O não atendimento às especificações de sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade implica que haverá uma diminuição expressiva na nota do trabalho.
- A ausência da documentação implica que haverá uma diminuição expressiva na nota do trabalho.
- A realização do trabalho prático com alunos de turmas diferentes implica que haverá uma diminuição expressiva na nota do trabalho.
- A inserção de palavras ofensivas nos arquivos e em qualquer outro material entregue implica que a nota final da parte do trabalho será igual a zero (0).
- Em caso de plágio, as notas dos trabalhos envolvidos serão zero (0).

Bom Trabalho!